

Assignment-9

Name : Anuska Nath

Roll : 002311001003

Dept : Information Technology

1. Implement the Sanon Fano coding techniques using Python.
2. Execute the implementations on text data and 8-bit grey-scale image data of your choice.
3. Calculate the input data size, compressed data size, compression ratio, average code length, entropy, and efficiency for each of the algorithm along with time to compress the text and image data.

REPORT: Shannon–Fano Coding Implementation

1. Aim

To implement Shannon–Fano Coding in Python and evaluate its performance on text and 8-bit grayscale image data by calculating compression metrics such as compression ratio, entropy, efficiency, and execution time.

2. Introduction to Shannon–Fano Coding

Shannon–Fano coding is a **lossless data compression technique** based on probability distribution of symbols. It assigns shorter binary codes to more frequent symbols and longer codes to less frequent ones.

Key Idea:

- Symbols are sorted in descending order of frequency
- The list is recursively divided into two groups with approximately equal probability
- Binary digits (0 and 1) are assigned at each split

This ensures efficient encoding while preserving all original information.

3. Methodology / Process

The implementation was carried out in the following steps:

Step 1: Import Required Libraries

```
import numpy as np
import pandas as pd
import time
from collections import Counter
from math import log2
from PIL import Image
```

Step 2: Implement Shannon–Fano Algorithm

```
def shannon_fano(symbols):
    codes = {symbol: "" for symbol in symbols}
    def split(symbol_list):
        if len(symbol_list) <= 1:
            return
        total = sum(freq for _, freq in symbol_list)
        acc = 0
        for i in range(len(symbol_list)):
            acc += symbol_list[i][1]
            if acc >= total / 2:
                break
        left = symbol_list[:i+1]
        right = symbol_list[i+1:]
        for symbol, _ in left:
            codes[symbol] += "0"
        for symbol, _ in right:
            codes[symbol] += "1"
        split(left)
        split(right)
    sorted_symbols = sorted(symbols.items(), key=lambda x: x[1], reverse=True)
    split(sorted_symbols)
    return codes
```

Step 3: Encode Data and Compute Metrics

```
def encode_data(data, codes):
    return "".join(codes[symbol] for symbol in data)
def entropy_calc(prob):
    return -sum(p * log2(p) for p in prob if p > 0)
```

Step 4: File Processing Function

```
def process_file(file_name, file_bytes):  
    # Handles both text and image inputs  
    # Converts image to grayscale and flattens pixels  
    # Applies Shannon–Fano coding  
    # Computes compression metrics
```

Step 5: Upload and Process Files

- Each file (1 text + 5 images) is uploaded individually
- The same function is applied to all inputs

Step 6: Generate Results Table

```
df = pd.DataFrame(results)
```

4. Input Data

Uploaded along with this document.

5. Output Results

	File Name	Type	Input Size (bits)	Compressed Size (bits)	Compression Ratio	Average Code Length	Entropy	Efficiency	Compression Time (s)
0	img5.jpg	Image	1392000	565540	2.461364	3.250230	3.017959	0.928537	0.052257
1	img4.png	Image	5050136	4655762	1.084707	7.375266	7.292273	0.988747	0.185729
2	img3.png	Image	2097152	1329500	1.577399	5.071640	4.954430	0.976889	0.088516
3	img2.jpg	Image	270816	255434	1.060219	7.545610	7.481733	0.991534	0.009851
4	text.txt	Text	488	248	1.967742	4.065574	3.913075	0.962490	0.000151
5	img1.jpg	Image	2824640	2791989	1.011695	7.907525	7.833412	0.990628	0.102652

6. Observations

- Text data achieved a higher compression ratio (1.96) due to lower entropy and higher redundancy.
- Images showed varying compression ratios depending on pixel distribution.
- Images with low entropy (e.g., img5.jpg) achieved better compression.
- Images with entropy close to 8 bits showed minimal compression.

- The average code length is slightly greater than entropy, satisfying theoretical expectations from Information Theory.
- Efficiency values close to 1 indicate near-optimal performance.
- Compression time increases with input size but remains efficient.

7. Conclusion

The implementation of Shannon–Fano coding successfully demonstrates lossless compression for both text and image data. The results show that compression efficiency depends on the entropy of the input data. Text data compresses better due to redundancy, while image data shows limited compression due to higher randomness. Overall, the algorithm performs efficiently and produces results consistent with theoretical expectations.